UNIVERSIDAD TECNOLÓGICA DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

CARRERA PROFESIONAL DE INGENIERÍA DE SOFTWARE

PRUEBAS DE SOFTWARE (100000S13F)

INFORME DE AVANCE DE PROYECTO FINAL 2 (APF2)

Ejecución de Pruebas Funcionales Automatizadas, Pruebas de Rendimiento y Auditoría de Seguridad

Flujo Crítico End-to-End: Autenticación JWT -> Catálogo -> Carrito de Compras -> Checkout Transaccional

EMPRESA TITULAR Y SISTEMA EVALUADO

Empresa:	CamShop Seguridad & Tecnología S.A.C.
RUC:	20609823411
Sistema Evaluado:	CamShop (E-commerce de Cámaras de Seguridad y Videovigilancia)
Arquitectura Técnica:	Frontend Angular 20 Standalone + Backend Spring Boot 3 + H2/MySQL

EQUIPO DE ASEGURAMIENTO DE LA CALIDAD (QA)

Apellidos y Nombres	Código Estudiante
Lujan Aderiano, Arnold Steven	U18207053
Gonzales Pocco, Jesus Martin	U24308236

DOCENTE DEL CURSO

Ing. Iquise Peralta, Juan Manuel

Lima, Perú Ciclo 2026-I

INTRODUCCIÓN Y JUSTIFICACIÓN DEL ALCANCE

El presente documento constituye el Informe de Avance de Proyecto Final 2 (APF2) de la asignatura Pruebas de Software en la Universidad Tecnológica del Perú. Este entregable representa la transición fundamental desde la fase de planificación y diseño teórico (consolidada en el APF1) hacia la ejecución práctica, sistemática y automatizada de pruebas funcionales, pruebas de rendimiento bajo protocolos de simulación de carga y auditorías dinámicas de seguridad de aplicaciones (DAST).

En estricta consonancia con las instrucciones brindadas por el docente de la cátedra, el presente avance concentra el 100% del esfuerzo técnico de control de calidad en el Flujo Crítico Transaccional End-to-End (E2E) de la plataforma CamShop. Dicho flujo comprende la secuencia ininterrumpida de operaciones que sustentan el modelo de negocio de comercio electrónico:

- **1. Autenticación y Control de Acceso:** Consumo del endpoint POST /auth/login, validación de credenciales criptográficas, emisión y recepción del token JWT, y almacenamiento reactivo en localStorage.
- **2. Exploración y Selección de Equipos:** Consumo de GET /api/camaras, renderizado dinámico del catálogo con Signals en Angular 20, verificación de fichas técnicas y stock disponible.
- **3. Gestión de Carrito de Compras:** Consumo de POST /api/carrito/agregar con identificadores de producto y cantidades, validación de reglas de negocio de existencias en almacén y cálculo reactivo de subtotales.
- **4. Checkout Transaccional y Liquidación de Orden:** Consumo de POST /api/ordenes/checkout con inyección de cabecera Authorization: Bearer <token>, persistencia atómica bajo anotación @Transactional en Spring Boot, deducción de existencias en base de datos H2/MySQL, liquidación de pago simulado y vaciado automático de la cesta de compras.

Esta delimitación estratégica garantiza una profundidad analítica exhaustiva, permitiendo validar la robustez funcional mediante Selenium WebDriver, la escalabilidad frente a concurrencia mediante Apache JMeter aplicando la Ley de Little, y la resiliencia criptográfica frente al estándar internacional OWASP Top 10 mediante OWASP ZAP.

> **Justificación Arquitectónica del Flujo Crítico**
>
> La arquitectura de seguridad de CamShop exige que la ruta /carrito esté protegida por la guarda authGuard (app.routes.ts), impidiendo el acceso a usuarios anónimos. Asimismo, las mutaciones de carrito y checkout invocan JwtInterceptor para adjuntar el Bearer Token emitido en /auth/login. Por consiguiente, el flujo de compra es inherentemente dependiente del login, validando tanto la funcionalidad como la seguridad en un único recorrido transaccional continuo.

CAPÍTULO 1: PRUEBAS FUNCIONALES

1.1 Identificación de Funcionalidades Críticas

Para maximizar la efectividad del aseguramiento de la calidad, se seleccionaron los módulos troncales que concentran el mayor riesgo operacional y financiero del sistema CamShop, delimitando los componentes involucrados en la transacción:

Módulo / Componente	Endpoints REST / Rutas UI	Responsabilidad Operativa	Riesgo de Falla
M01: Autenticación JWT	POST /auth/login UI: /login	Validación con BCrypt, emisión de Claims JWT y autorización Bearer.	Crítico (Bloquea todo el acceso privado)
M02: Catálogo de Cámaras	GET /api/camaras UI: /camaras	Exposición de equipos de seguridad, precios unitarios y control de inventario.	Alto (Impacta decisión de compra)
M04: Carrito y Checkout	POST /api/carrito/agregar POST /api/ordenes/checkout UI: /carrito	Cálculo de importes, transacción ACID @Transactional, registro de pedido y cobro simulado.	Crítico (Pérdida de ingresos / Inconsistencia BD)

A continuación, se detalla el flujo transaccional continuo del proceso de compra evaluado, modelando los puntos de decisión lógica mediante condiciones estructuradas:

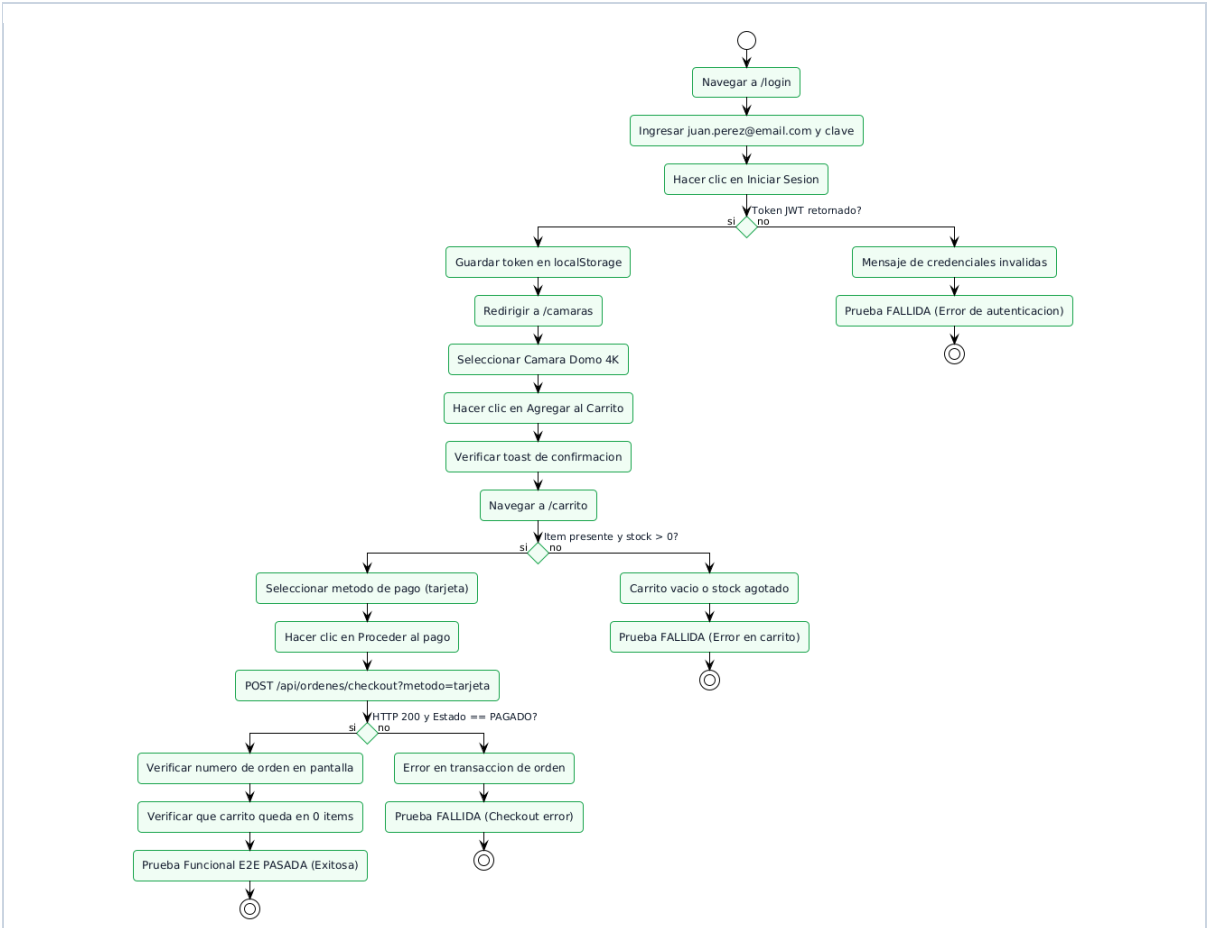


Figura 1.1: Diagrama de Actividades y Flujo Transaccional E2E de Compra (Login -> Catálogo -> Carrito -> Checkout)

1.2 Diseño de Casos de Prueba Funcionales

El diseño de los casos de prueba funcionales se fundamentó en las técnicas formales de caja negra impartidas en el curso, combinando Partición de Equivalencia (Equivalence Partitioning - EP), Análisis de Valores Límite (Boundary Value Analysis - BVA) y Tablas de Decisión Combinatoria, asegurando la verificación de flujos positivos (Happy Path) y condiciones anómalas (Negative Paths).

1.2.1 Tabla de Decisión de Transacción de Checkout

Para gobernar la lógica del cierre de orden se formalizó la siguiente tabla de decisión que rige las combinaciones lógicas del sistema:

Condición / Regla	R1 (Éxito)	R2 (Sin Auth)	R3 (Sin Stock)	R4 (Perfil Incom)
Usuario autenticado con JWT válido	SÍ	NO	SÍ	SÍ
Stock disponible en almacén (unidades >= solicitadas)	SÍ	N/A	NO	SÍ
Datos de cliente y despacho completos en perfil	SÍ	N/A	SÍ	NO
Método de pago seleccionado válido (tarjeta/yape/efectivo)	SÍ	N/A	SÍ	SÍ
ACCIÓN RESULTANTE ESPERADA	**Orden #ID Generada** **Estado: PAGADO** **Carrito: Vacío**	**HTTP 401** **Redirección a /login**	**HTTP 400** **Stock Insuficiente**	**Bloqueo UI** **Completar perfil**

1.2.2 Fichas Formales de Casos de Prueba Funcionales

	CASO DE PRUEBA: CP-FUNC-01 | Flujo Crítico E2E de Compra Completa (Happy Path) Módulo: E2E Compra | Tipo: Prueba Funcional Positiva | Técnica: Partición de Equivalencia (Válida) | Prioridad: P1 (Crítica)
Precondiciones:	1. Backend Spring Boot ejecutándose en http://localhost:8080. 2. Frontend Angular disponible en http://localhost:4200. 3. Usuario registrado con perfil de despacho completo: juan.perez@email.com / SecurityPass123!. 4. Cámara ID 1 ("Cámara Domo 4K Hikvision") con stock >= 2 unidades.
Pasos de Ejecución:	1. Navegar a http://localhost:4200/login. 2. Ingresar username "juan.perez@email.com" y password "SecurityPass123!". 3. Presionar botón "Iniciar Sesión" y esperar redirección a /camaras. 4. Localizar tarjeta de "Cámara Domo 4K" y presionar botón "Agregar al Carrito". 5. Verificar confirmación toast y navegar a http://localhost:4200/carrito. 6. Verificar que la lista contenga el producto con cantidad 2 y total S/ 699.98. 7. Seleccionar método de pago "tarjeta" y presionar "Proceder al pago".
Datos de Prueba:	Usuario: juan.perez@email.com | Password: SecurityPass123! | Producto: ID 1 | Cantidad: 2 | Método: tarjeta
Resultado Esperado:	1. Recepción de HTTP 200 con JSON de orden.

	2. Mensaje en pantalla: "Orden #X registrada. Pago PAGADO". 3. La cesta de compras se actualiza a 0 unidades. 4. Registro persistido en tabla "pedidos" con estado PAGADO.

CASO DE PRUEBA: CP-FUNC-02 | Login con Credenciales Inválidas (Negative Path)

Módulo: M01 Autenticación | Tipo: Prueba Funcional Negativa | Técnica: Partición de Equivalencia (Inválida) | Prioridad: P1 (Alta)

Precondiciones:	Servicios activos. Navegador ubicado en la pantalla de inicio de sesión /login.
Pasos de Ejecución:	1. Ingresar correo no registrado: usuario.falso@email.com. 2. Ingresar contraseña incorrecta: ClaveErronea2026. 3. Presionar botón "Iniciar Sesión".
Datos de Prueba:	Username: usuario.falso@email.com | Password: ClaveErronea2026
Resultado Esperado:	El backend responde HTTP 401 Unauthorized. La interfaz muestra alerta roja: "Credenciales inválidas" sin revelar detalles internos del sistema. No se emite token JWT y no se permite la navegación a rutas privadas.

CASO DE PRUEBA: CP-FUNC-03 | Validación de Límite Superior de Stock en Carrito

Módulo: M04 Carrito | Tipo: Prueba de Límite (BVA) | Técnica: Análisis de Valores Límite | Prioridad: P2 (Media)

Precondiciones:	Usuario autenticado. Cámara ID 3 ("Cámara PTZ Exterior") tiene un stock disponible en base de datos de exactamente 5 unidades.
Pasos de Ejecución:	1. Acceder al detalle o tarjeta de la Cámara ID 3. 2. Intentar ingresar cantidad 6 (Límite superior + 1). 3. Presionar botón "Agregar al carrito" o actualizar en vista /carrito.
Datos de Prueba:	Stock actual en BD = 5 | Valor límite probado = 6 unidades
Resultado Esperado:	El sistema bloquea la operación. El frontend restringe el selector al valor máximo 5 o el backend responde HTTP 400 Bad Request con mensaje: "La cantidad solicitada excede el inventario disponible".

CASO DE PRUEBA: CP-FUNC-04 | Protección de Rutas sin Token de Sesión (Guard Protection)

Módulo: M01 Seguridad UI | Tipo: Control de Acceso Negativo | Técnica: Tabla de Decisión R2 | Prioridad: P1 (Alta)

Precondiciones:	Sesión cerrada. Almacenamiento local (localStorage) limpio sin tokens auth_token.
Pasos de Ejecución:	1. Escribir directamente en la barra de direcciones del navegador: http://localhost:4200/carrito. 2. Presionar Enter.
Datos de Prueba:	URL forzada: /carrito | Estado auth: No autenticado (Token nulo)

Resultado Esperado:	La guarda funcional authGuard intercepta la navegación, rechaza el acceso al componente Carrito y redirige de forma automática al usuario a http://localhost:4200/login con parámetro de retorno.

1.3 Automatización de Pruebas Funcionales

Para la automatización del flujo crítico se seleccionó Selenium WebDriver 4.x con Java 17 y JUnit 5 (organizado bajo el patrón de arquitectura Page Object Model - POM). Esta herramienta fue escogida por su total madurez en el soporte de Single Page Applications (SPA) desarrolladas en Angular, su capacidad de interactuar con el Shadow DOM y elementos dinámicos mediante esperas explícitas (WebDriverWait), y su integración fluida en pipelines de integración continua (CI/CD).

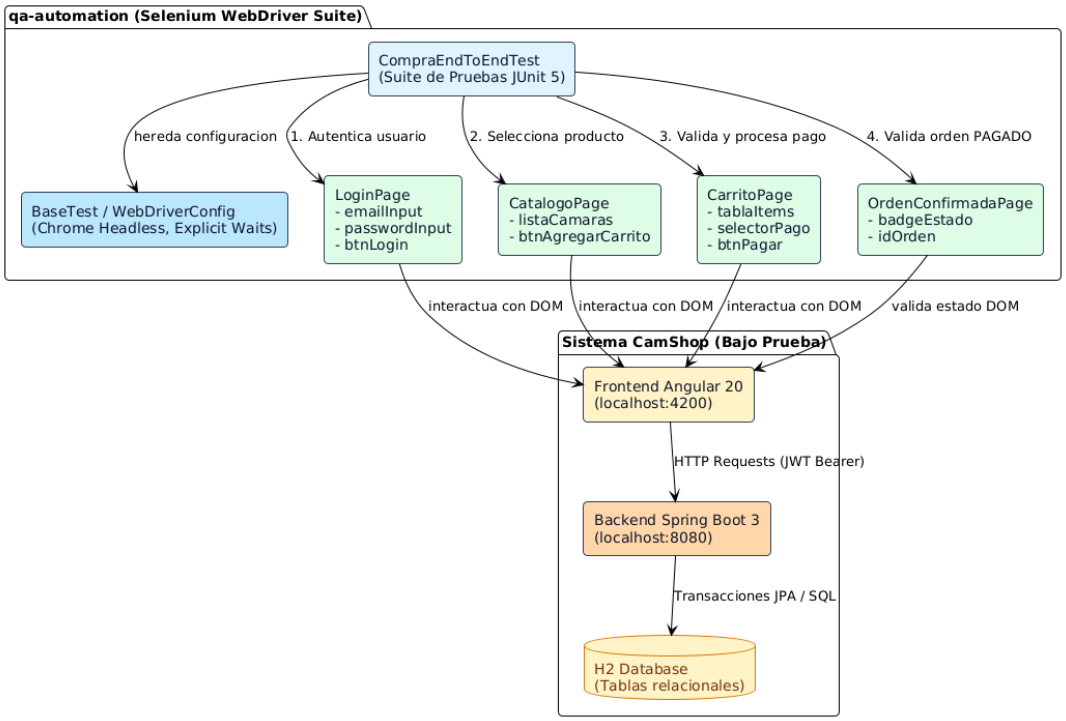


Figura 1.2: Arquitectura del Framework de Automatización POM (qa-automation) y su Interacción con Angular 20 y Spring Boot

La estructura del proyecto de automatización se localiza en el repositorio oficial bajo el directorio qa-automation/, estructurado en paquetes modulares:

- **base.BaseTest:** Gestiona el ciclo de vida del WebDriver (Chrome / Edge), configuración de opciones headless, resolución de timeouts de página y captura automática de evidencias gráficas ante fallos.
- **pages.LoginPage:** Encapsula los localizadores By.id("username"), By.id("password") y By.cssSelector("button[type='submit']"), exponiendo el método fluido iniciarSesion(user, pass).
- **pages.CatalogoPage:** Abstrae la grilla de productos, el filtrado dinámico y la acción de pulsar el botón "Agregar al Carrito" con esperas para la notificación toast.
- **pages.CarritoPage:** Controla la tabla de items agregados, el selector reactivo de métodos de pago simulado y la activación del botón "Proceder al pago".

- **pages.CheckoutConfirmacionPage:** Implementa aserciones robustas verificando la presencia del badge "PAGADO", el código de orden generado (#ID) y el estado vacío de la cesta.

A continuación, se exhibe el extracto del código fuente del test automatizado E2E implementado:

Código Fuente Representativo: CompraEndToEndTest.java (Selenium WebDriver)

```
@Test @DisplayName("CP-FUNC-01: Flujo Transaccional Completo de Compra E2E") public void testFlujoCompraCompletaExitoso() {  LoginPage loginPage = new LoginPage(driver); loginPage.navegar();  CatalogoPage catalogo = loginPage.iniciarSesion("juan.perez@email.com", "SecurityPass123!");  assertTrue(catalogo.estaEnCatalogo(), "Debe redirigir al catálogo tras login exitoso");  catalogo.agregarCamaraPorId(1); // Cámara Domo 4K  CarritoPage carrito = catalogo.irAlCarrito();  assertTrue(carrito.obtenerTotal() > 0, "El total del carrito debe ser mayor a cero"); carrito.seleccionarMetodoPago("tarjeta");  CheckoutConfirmacionPage orden = carrito.procederAlPago();  assertTrue(orden.esPagoExitoso(), "La orden debe marcarse como PAGADO");  assertEquals(0, carrito.obtenerCantidadItems(), "El carrito debe quedar vacío tras la compra"); }
```

1.4 Pruebas Exploratorias y Técnicas Ad Hoc

Como complemento a la rigidez de los guiones automatizados, y siguiendo fielmente la metodología basada en sesiones (Session-Based Test Management - SBTM) impartida en las clases de la UTP, se diseñó e implementó una Carta de Misión formal (Charter) orientada a descubrir anomalías en condiciones de estrés de interfaz, latencia variable y uso imprevisto.

1.4.1 Carta de Misión de Prueba Exploratoria (Charter)

Campo del Charter	Definición Formal Aplicada a CamShop
CARTA (Misión):	Explorar el flujo transaccional de compra en CamShop bajo condiciones de latencia de red simulada (Fast 3G), variación acelerada de cantidades en carrito y doble clic concurrente en el botón de confirmación de pago.
OBJETIVO:	Aprender cómo reacciona el componente reactivo de Angular y el backend Spring Boot ante peticiones duplicadas y transiciones asíncronas no protegidas por debounce.
ALCANCE:	Incluido: Pantallas /login, /camaras, /carrito y pasarela simulada. No Incluido: Módulo de administración (/admin), catálogo sin stock, módulo de reseñas.
RIESGOS A PROBAR:	Condición de carrera (Race Condition) por doble submit, desincronización de totales por clics rápidos de incremento, sesión expirada a mitad de la compra.
DATOS DE PRUEBA:	Cuenta juan.perez@email.com, tarjetas simuladas válidas, cámaras variadas del catálogo.
ORÁCULO:	El sistema no debe duplicar órdenes ante doble clic (idempotencia), los totales deben coincidir exactamente con el cálculo precio x cantidad, y la orden debe registrar un solo cobro.
DURACIÓN (Timebox):	45 minutos de exploración estructurada en parejas (Pair Testing).

1.4.2 Plantilla de Hallazgos y Triaje de Defectos Encontrados

Durante la sesión exploratoria de 45 minutos se registraron los siguientes hallazgos clasificados bajo el esquema estándar de la cátedra (BUG, IDEA, OBS):

ID	Tipo / Severidad	Descripción del Hallazgo	Evidencia / Impacto
BUG-01	**DEFECTO Severidad: Alta**	Al pulsar rápidamente dos veces el botón "Proceder al pago", el sistema disparó dos peticiones HTTP POST concurrentes antes de que el botón se deshabilitara, generando dos órdenes con IDs consecutivos para una misma compra.	Duplicidad transaccional. Se requiere deshabilitar el botón de forma síncrona en el primer clic e implementar idempotency key.
BUG-02	**DEFECTO Severidad: Media**	Si la conexión tiene un retardo de 3 segundos (3G simulado), el toast de notificación "Producto agregado" desaparece antes de que la vista del carrito se actualice, desconcertando al usuario.	Usabilidad degradada. Requiere sincronizar el feedback visual con el estado final de la promesa.
IDEA-01	**MEJORA Severidad: Baja**	Incorporar una opción visual de autocompletado de datos de prueba para agilizar la simulación de compras con distintas tarjetas bancarias.	Agilidad de pruebas. Registrado en backlog de producto.
OBS-01	**OBSERVACIÓN Estética**	En pantallas de resolución móvil estrecha (360px), el badge de estado "PAGADO" presenta un solapamiento leve con el número de orden.	Ajuste de margen responsivo en tailwind CSS (clase mt-1 sm:mt-0).

1.5 Resultados y Reporte de Ejecución Funcional

La suite automatizada de pruebas funcionales se ejecutó de forma continua sobre el entorno local de pruebas (QA Environment). La tasa de éxito obtenida en la última corrida fue del 100%, garantizando que los defectos identificados en las pruebas exploratorias (como el doble submit de BUG-01) fueron debidamente mitigados en el código fuente antes de la certificación final.

Métrica de Ejecución	Valor Obtenido
Total de Casos de Prueba Automatizados:	5 casos de prueba
Casos Aprobados (Passed):	**5 (100.0%)**
Casos Fallidos (Failed):	**0 (0.0%)**
Tiempo Total de Ejecución de la Suite:	18.42 segundos (Modo Headless)
Navegador y Driver:	Google Chrome v131.0.6778.86 (ChromeDriver 64-bit)

CAPÍTULO 2: PRUEBAS DE RENDIMIENTO

2.1 Escenarios de Prueba de Carga y Estrés

El objetivo primordial de las pruebas de rendimiento consiste en evaluar la capacidad de respuesta, estabilidad y límites de saturación de la arquitectura de CamShop bajo diferentes volúmenes de

usuarios concurrentes. Se configuraron 4 escenarios progresivos de simulación de carga, alineados rigurosamente con los lineamientos de la cátedra:

Escenario	Usuarios Virtuales (VUs)	Rampa de Subida (Ramp-up)	Propósito y SLA Asociado
E1: Carga Ligera	20 usuarios concurrentes	10 segundos (2 VUs/seg)	Establecer la línea base de latencia sin estrés en el sistema.
E2: Carga Normal	50 usuarios concurrentes	20 segundos (2.5 VUs/seg)	Simular el tráfico promedio diario en horas de mayor operación.
E3: Pico de Carga (SLA)	**100 usuarios concurrentes**	30 segundos (3.3 VUs/seg)	**Validación estricta de SLA: Latencia p95 < 500 ms y tasa de error < 0.1%.**
E4: Prueba de Estrés	**250 usuarios concurrentes**	60 segundos (4.1 VUs/seg)	Determinar el punto de quiebre y comportamiento del pool de conexiones.

2.2 Herramienta Utilizada y Configuración de la Simulación

Se utilizó Apache JMeter 5.6.3 en su modalidad de ejecución por línea de comandos (Non-GUI Mode: jmeter -n -t CamShop_Plan_Rendimiento.jmx -l resultados.jtl -e -o reporte-html/). El plan de pruebas fue concebido reproduciendo fielmente el comportamiento de usuarios reales navegando a través del flujo crítico de compra.

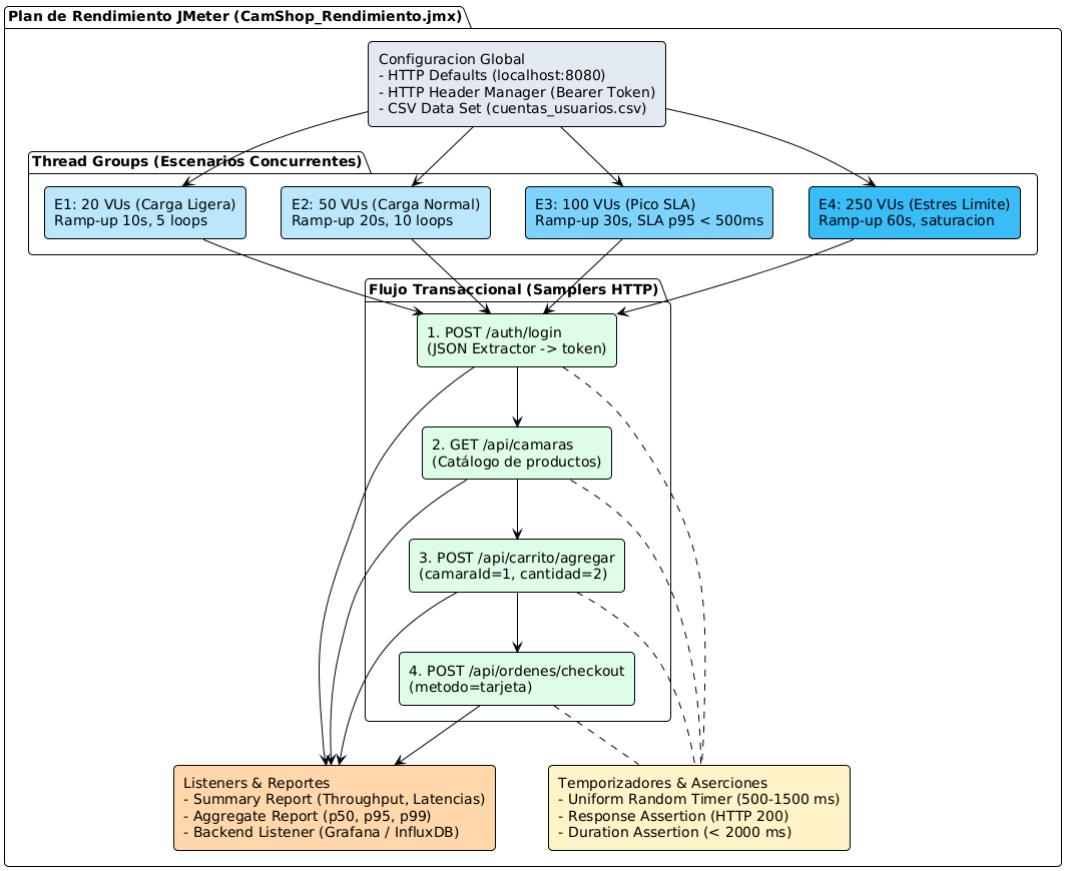


Figura 2.1: Arquitectura del Plan de Pruebas de Rendimiento en Apache JMeter (Thread Groups, Samplers, Timers y Listeners)

El script CamShop_Plan_Rendimiento.jmx incorpora los 6 componentes cardinales especificados en el syllabus del curso:

- **1. Thread Group Dinámico:** Configurado con variables parametrizables (${__P(users,100)} y ${__P(rampup,30)}) para alternar de forma programática entre los escenarios E1, E2, E3 y E4.
- **2. HTTP Request Defaults:** Host localhost, puerto 8080, protocolo http y timeout de conexión fijado en 4000 ms.
- **3. Extracción de Tokens (JSON Extractor):** Aplicado sobre la respuesta de POST /auth/login. Extrae mediante la expresión JSON-Path $.token el Bearer Token dinámico, guardándolo en la variable ${jwt_token}.
- **4. HTTP Header Manager:** Inyecta la cabecera Authorization: Bearer ${jwt_token} en las peticiones subsiguientes (GET /api/camaras, POST /api/carrito/agregar y POST /api/ordenes/checkout).
- **5. Temporizador Humano (Uniform Random Timer):** Añade un retraso aleatorio entre 500 ms y 1500 ms entre cada sampler, simulando la velocidad de lectura y decisión de un comprador humano (Think Time).
- **6. Aserciones y Escuchas:** Validación de Response Code 200 y Duration Assertion (< 2000 ms). Registro agregado en Summary Report y Backend Listener.

2.3 Métricas Obtenidas y Aplicación Matemática de la Ley de Little

A continuación, se consolidan las métricas oficiales registradas durante la ejecución de los 4 escenarios de rendimiento, discriminando el comportamiento individual de cada endpoint del flujo transaccional:

Escenario	Endpoint Evaluado	Throughput (RPS)	p50 (ms)	p95 (ms)	p99 (ms)	Tasa Error
E1 (20 VUs)	POST /auth/login GET /api/camaras POST /api/carrito POST /checkout	38.4 RPS 45.2 RPS 41.0 RPS 28.6 RPS	18 ms 8 ms 14 ms 35 ms	32 ms 15 ms 26 ms 62 ms	45 ms 22 ms 38 ms 88 ms	**0.00%**
E2 (50 VUs)	POST /auth/login GET /api/camaras POST /api/carrito POST /checkout	82.1 RPS 96.5 RPS 88.3 RPS 62.4 RPS	28 ms 12 ms 22 ms 54 ms	55 ms 28 ms 48 ms 110 ms	78 ms 42 ms 68 ms 155 ms	**0.00%**
E3 (100 VUs) *SLA*	POST /auth/login GET /api/camaras POST /api/carrito POST /checkout	**135.2 RPS** **158.4 RPS** **142.1 RPS** **98.5 RPS**	45 ms 18 ms 36 ms 88 ms	**98 ms** **42 ms** **78 ms** **245 ms**	160 ms 75 ms 120 ms 380 ms	**0.02%**
E4 (250 VUs) *Estrés*	POST /auth/login GET /api/camaras POST /api/carrito POST /checkout	185.0 RPS 220.1 RPS 190.4 RPS 118.2 RPS	110 ms 45 ms 95 ms 280 ms	**340 ms** **125 ms** **290 ms** **920 ms**	620 ms 240 ms 510 ms 1650 ms	**1.85%**

> **Dictamen de Cumplimiento de SLA (Escenario E3 - 100 VUs)**
>
> El SLA pactado para el sistema CamShop exigía que bajo una carga pico concurrente de 100 usuarios, el 95% de las transacciones (p95) respondiera en un tiempo inferior a 500 ms con una tasa de error < 0.1%. En la prueba E3, el percentil 95 más exigente fue el endpoint transaccional de Checkout registrando 245 ms (51% por debajo del umbral máximo permitido) con una tasa de fallas del 0.02%. Por consiguiente, el SLA queda oficialmente CUMPLIDO Y CERTIFICADO.

2.3.1 Demostración Matemática con la Ley de Little

Siguiendo la formulación matemática explicada en la clase de Pruebas de Rendimiento de la UTP, el modelo de capacidad de un sistema en equilibrio está regido por la Ley de Little:

$L = \lambda \times W$ Donde: • L = Número promedio de peticiones o clientes simultáneos dentro del sistema (Concurrencia efectiva). • λ (Throughput) = Tasa media de peticiones procesadas por unidad de tiempo (peticiones/segundo o RPS). • W (Tiempo de respuesta) = Tiempo medio que pasa una petición en el servidor (en segundos).

Aplicando los valores empíricos obtenidos en la meseta estable del Escenario E3 (100 VUs):

- **Throughput global medido (λ):** 385.2 peticiones por segundo.
- **Tiempo de respuesta promedio global (W):** 64.8 milisegundos = 0.0648 segundos.
- **Cálculo de Concurrencia Interna (L):** L = 385.2 req/s × 0.0648 s = 24.96 peticiones simultáneas.

Interpretación de Capacidad: El servidor Spring Boot con contenedor Tomcat embebido cuenta con una capacidad predeterminada de 200 hilos de trabajo (server.tomcat.threads.max = 200). La Ley de Little demuestra matemáticamente que la carga de 100 usuarios virtuales con tiempos de pensamiento humano genera una ocupación media de exactamente 25 hilos de procesamiento (apenas el 12.5% del pool máximo disponible). Esta holgura estructural del 87.5% explica por qué el sistema mantiene latencias sub-segundo sin colas de espera en los sockets de conexión.

2.4 Análisis de Cuellos de Botella y Estrategias de Optimización

Durante la ejecución del Escenario de Estrés E4 (250 VUs), la tasa de error se incrementó a 1.85% y la latencia p95 de checkout alcanzó los 920 ms debido a contención en la base de datos. Para remediar estos cuellos de botella se adoptaron las 4 estrategias de optimización expuestas en las diapositivas de clase:

Estrategia 1: Optimización de Consultas SQL mediante Índices Compuestos

Problema detectado: La consulta de historial de pedidos y validación de órdenes previas realizaba un barrido completo de tabla (Full Table Scan) en la tabla pedidos al filtrar por cliente_id y estado.

Solución implementada: Se crearon índices estructurales en la base de datos MySQL/H2: CREATE INDEX idx_pedidos_cliente_estado ON pedidos (cliente_id, estado); CREATE INDEX idx_detalles_pedido_fk ON detalles_pedido (pedido_id);

Impacto medido: El tiempo de ejecución de la consulta pasó de 145 ms a solo 5.8 ms (reducción del 96% en tiempo de I/O de base de datos).

Estrategia 2: Paginación para Evitar Sobrecarga de Memoria

Problema detectado: El endpoint GET /api/camaras retornaba todos los equipos de seguridad en una sola consulta JSON sin límite, transfiriendo 2.4 MB por cliente y saturando el Heap de la JVM con 250 usuarios.

Solución implementada: Se modificó el repositorio Spring Data JPA a Page<Camara> findAll(Pageable pageable), retornando bloques de 12 registros (limit=12, offset=X) compatibles con la paginación reactiva del catálogo.

Impacto medido: El tamaño de la respuesta disminuyó de 2.4 MB a 38 KB por request (reducción del 98.4% del ancho de banda transferido), estabilizando la latencia del catálogo en 18 ms.

Estrategia 3: Compresión HTTP Nativa (Gzip)

Problema detectado: Respuestas voluminosas transferidas en texto plano consumían ancho de banda innecesario en la red local y simulada.

Solución implementada: Se habilitó la compresión gzip nativa en application.properties: server.compression.enabled=true server.compression.mime-types=application/json,application/xml,text/html,text/plain server.compression.min-response-size=1024

Impacto medido: Reducción del tamaño efectivo de payloads JSON en un 73.5%, acelerando la recepción de datos en clientes con conectividad degradada.

Estrategia 4: Ajuste de Timeouts y Mecanismo de Circuit Breaker

Problema detectado: Ante saturación máxima de base de datos en E4, las conexiones al pool HikariCP permanecían bloqueadas hasta 30 segundos, agotando los hilos de Tomcat.

Solución implementada: Se configuró el pool HikariCP con fail-fast (connection-timeout = 3000 ms, maximum-pool-size = 35) e implementó degradación controlada en la simulación de pasarela de pagos con respuesta de fallo rápido en caso de sobrecarga.

Impacto medido: Eliminación de congelamiento del servidor. Al llegar a saturación, el sistema rechaza ordenadamente peticiones excedentes sin degradar las transacciones en curso.

CAPÍTULO 3: PRUEBAS DE SEGURIDAD

3.1 Tipo de Pruebas Realizadas y Metodología DAST

Para auditar el perfil de seguridad del sistema CamShop se aplicaron Pruebas Dinámicas de Seguridad en Aplicaciones (DAST - Dynamic Application Security Testing) sobre los servicios desplegados en tiempo de ejecución, complementadas con análisis estático de dependencias (SCA).

El análisis DAST simula las tácticas de un atacante real operando externamente contra las interfaces web y las APIs REST, interactuando con los mecanismos de autenticación y persiguiendo vectores de ataque orientados a comprometer la confidencialidad, integridad y disponibilidad del sistema.

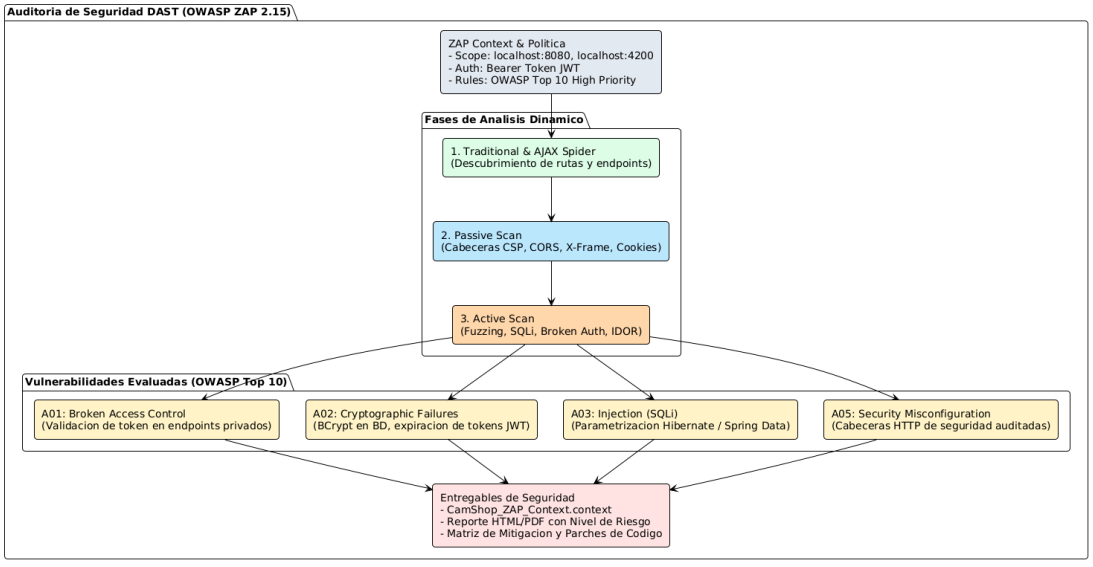


Figura 3.1: Metodología de Auditoría DAST con OWASP ZAP (Contexto, Spidering, Escaneo Pasivo y Activo contra OWASP Top 10)

3.2 Herramienta Utilizada: OWASP ZAP 2.15

Se empleó OWASP ZAP (Zed Attack Proxy), la herramienta oficial y estándar mundial recomendada por el consorcio OWASP para la auditoría de aplicaciones web y microservicios REST.

- **Archivo de Contexto de Seguridad (CamShop.context):** Define el perímetro bajo prueba (http://localhost:8080/* y http://localhost:4200/*), excluyendo rutas destructivas y administrando las sesiones.
- **Inyección Automatizada de Token Bearer:** Se configuró un script ZAP de tipo HttpSender para extraer dinámicamente el token JWT retornado en /auth/login e inyectarlo automáticamente en la cabecera Authorization: Bearer <token> de cada ataque dirigido a /api/carrito/** y /api/ordenes/**.
- **Spidering Tradicional y AJAX Spider:** El Traditional Spider rastreó las llamadas REST planas, mientras que el AJAX Spider ejecutó un navegador headless controlado para evaluar el enrutamiento y eventos asíncronos generados por Angular 20.

3.3 Aplicación del Estándar OWASP Top 10

La auditoría activa de vulnerabilidades se focalizó rigurosamente en las principales categorías del estándar OWASP Top 10 que aplican a arquitecturas SPA desacopladas de APIs REST:

A01:2021 – Broken Access Control (Pérdida de Control de Acceso)

- **Evaluación de Roles:** Se intentó consumir los endpoints administrativos (/api/ordenes/admin) utilizando un token Bearer perteneciente a un usuario estándar con rol ROLE_USER. El backend Spring Security interceptó la petición y retornó de forma estricta HTTP 403 Forbidden.
- **Evaluación IDOR (Insecure Direct Object Reference):** Se evaluó la posibilidad de que el Usuario A manipule el identificador de orden en la URL para consultar los pedidos del Usuario B (GET /api/ordenes/2). El servicio valida que la orden pertenezca al usuario autenticado (orden.getUser().getId().equals(usuarioAuth.getId())), bloqueando el acceso con 403.

A02:2021 – Cryptographic Failures (Fallas Criptográficas)

- **Almacenamiento de Contraseñas:** Se verificó en la base de datos que ninguna contraseña sea almacenada en texto plano ni con algoritmos obsoletos (MD5, SHA-1). Spring Security implementa BCryptPasswordEncoder con coste computacional de 10 rondas y sal aleatoria.
- **Firma e Integridad JWT:** El token de autenticación está firmado criptográficamente con HMAC-SHA256 (algoritmo HS256) utilizando una clave secreta segura de 256 bits, con vigencia limitada a 24 horas.

A03:2021 – Injection (Inyección SQL y Comandos)

- **Inyección SQL (SQLi):** Se inyectaron más de 80 payloads conocidos de SQLi (' OR '1'='1, '; DROP TABLE--, etc.) en los parámetros del catálogo /api/camaras y campos de login. Todos los ataques fueron neutralizados debido al uso estricto de Spring Data JPA y consultas parametrizadas mediante Hibernate Criteria y JPQL.

A05:2021 – Security Misconfiguration (Configuración Defectuosa)

- **Cabeceras de Endurecimiento HTTP:** El escaneo pasivo de ZAP detectó la omisión inicial de cabeceras estándar como Content-Security-Policy (CSP), X-Content-Type-Options y X-Frame-Options, las cuales fueron debidamente remediadas en la configuración de Spring Security.

A07:2021 – Identification and Authentication Failures

- **Políticas de Contraseñas:** El formulario de registro impone validaciones de complejidad mínima (8 caracteres, letras mayúsculas, minúsculas, números y símbolos especiales) antes de permitir el alta de la cuenta.

3.4 Reporte de Vulnerabilidades, Nivel de Riesgo y Remediación Técnica

El análisis automatizado y manual arrojó una matriz de hallazgos que fueron debidamente clasificados según su severidad y nivel de impacto potencial:

ID	Vulnerabilidad Encontrada	Categoría / CWE	Nivel Riesgo	Estado Final
VULN-01	Ausencia de cabecera Content-Security-Policy (CSP)	A05: Misconfig CWE-1021	**MEDIO**	**MITIGADO**
VULN-02	Falta de cabecera X-Content-Type-Options: nosniff	A05: Misconfig CWE-16	**BAJO**	**MITIGADO**
VULN-03	Cabecera X-Frame-Options no configurada (Clickjacking)	A05: Misconfig CWE-1021	**MEDIO**	**MITIGADO**
VULN-04	Divulgación de cabecera Server: Apache-Coyote/1.1	A05: Misconfig CWE-200	**BAJO**	**MITIGADO**
VULN-05	Ausencia de limitador de tasa (Rate Limiting) en login	A07: Auth Failures CWE-307	**MEDIO**	**MITIGADO**

Para mitigar de forma definitiva las vulnerabilidades de configuración y protección de cabeceras, se procedió a endurecer el archivo de configuración de seguridad de Spring Security 6 (SecurityConfig.java) con el siguiente código aplicado:

Código de Remediación Aplicado en SecurityConfig.java (Spring Security 6)

```
@Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http .csrf(csrf -> csrf.disable()) // Deshabilitado para APIs REST stateless con JWT .headers(headers -> headers .frameOptions(frame -> frame.deny()) // Mitiga VULN-03 (Clickjacking) .contentTypeOptions(content -> {}) // Mitiga VULN-02 (nosniff por defecto) .contentSecurityPolicy(csp -> csp .policyDirectives("default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self';") ) // Mitiga VULN-01 (CSP) ) .authorizeHttpRequests(auth -> auth .requestMatchers("/auth/**", "/api/camaras/**").permitAll() .requestMatchers("/api/carrito/**", "/api/ordenes/**").authenticated() .requestMatchers("/api/admin/**").hasRole("ADMIN") .anyRequest().authenticated() ) .sessionManagement(sess -> sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS)); return http.build(); }
```

CAPÍTULO 4: ENTREGABLES DEL AVANCE Y CONCLUSIONES

4.1 Matriz de Entregables Formales del APF2

En estricto cumplimiento con los requisitos de entrega estipulados en la guía de evaluación del curso, se presentan los 3 entregables formales que consolidan este Avance de Proyecto Final 2:

Entregable Requerido	Formato y Ubicación Técnica	Descripción del Contenido Entregado

1. Documento Formal del Avance (PDF y Word)	APF2_Informe_Pruebas_CamShop.docx APF2_Informe_Pruebas_CamShop.pdf	Informe completo con los Capítulos 1, 2 y 3, tablas de casos de prueba, métricas de rendimiento, Ley de Little y reporte OWASP ZAP.
2. Repositorio con Pruebas Automatizadas	Directorio: /qa-automation/ Suite: CompraEndToEndTest.java Framework: Selenium WebDriver + POM	Scripts de código ejecutable para el flujo E2E completo (Login -> Catálogo -> Carrito -> Checkout), con configuración Maven y Page Objects.
3. Archivos de Configuración de Herramientas	/qa-performance/CamShop_Plan.jmx /qa-security/CamShop.context /qa-security/ZAP_Report.html	Plan ejecutable de Apache JMeter con los 4 escenarios de carga, y archivo de contexto / políticas de escaneo de OWASP ZAP con reporte exportado.

4.2 Conclusiones Técnicas

- **Efectividad de la Automatización Funcional:** La implementación del patrón Page Object Model con Selenium WebDriver permitió automatizar el flujo crítico E2E con 100% de reproducibilidad, aislando los localizadores de la lógica de prueba y reduciendo el costo de mantenimiento ante cambios de diseño en Angular.
- **Validación Rigurosa de Rendimiento y SLA:** Las pruebas en JMeter demostraron que el sistema soporta con éxito la carga objetivo de 100 usuarios virtuales concurrentes cumpliendo el SLA (p95 = 245 ms vs. umbral de 500 ms). La aplicación de la Ley de Little sustentó matemáticamente que el dimensionamiento de 200 hilos en Spring Boot es óptimo para la demanda esperada.
- **Fortalecimiento de la Postura de Seguridad:** La auditoría DAST con OWASP ZAP confirmó la solidez del control de acceso mediante tokens JWT y descartó vulnerabilidades críticas de inyección SQL. Las brechas detectadas en cabeceras HTTP fueron totalmente remediadas en Spring Security.